SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

Assessment Tool Weightage: 30%

QUESTIONS: 10 **Total Marks:50**

Annexure A

Debugging or Optimization task

Code of Conduct:

I Atchaya Chandran R(192521394) *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Q1. Deadlock Debugging (Code Trace)

1. Root Cause Identification

The deadlock is caused by a **Circular Wait** condition resulting from asymmetric resource acquisition order across threads.

PDF

- **Hold and Wait:** Process ⬚⬚$_1$acquires lock ⬚⬚$_1$and holds it while waiting to acquire ⬚⬚$_2$. Process ⬚⬚$_2$acquires lock ⬚⬚$_2$and holds it while waiting to

acquire R_1.

PDF+ 1

- **Circular Wait:** Process P_1 holds R_1 and waits for R_2 (which is held by P_2), while process P_2 holds R_2 and waits for R_1 (which is held by P_1). The artificial delay (wait(100 ms)) guarantees that both processes acquire their initial lock before attempting to claim the second.

PDF+ 1

2. Optimization (Deadlock Elimination)

To eliminate deadlock without altering core process logic, enforce a **Global Lock Ordering Hierarchy** (Resource Hierarchy Solution). Both processes must acquire resources in the exact same numerical/lexicographical order.

C

```
// Process P1

lock(R1);

wait(100 ms);

lock(R2);

critical_section();

unlock(R2);

unlock(R1);


// Process P2 (Optimized: lock ordering changed to match P1)

lock(R1); // Replaced lock(R2) with lock(R1)

wait(100 ms);

lock(R2); // Replaced lock(R1) with lock(R2)

critical_section();

unlock(R2);

unlock(R1);
```

Justification: By standardizing the locking order ($R_1 \rightarrow R_2$), P_2 cannot acquire R_2 until it successfully acquires R_1. This breaks the Circular Wait condition entirely while preserving process execution logic.

Q2. CPU Utilization Problem

1. Root Cause Identification
The rapid state transition pattern (ready →waiting →ready →waiting) combined with low CPU utilization (30%) indicates **Thrashing** or **I/O-Bound Bottleneck/Priority Inversion**.

PDF

- Processes are continuously being moved to the waiting state to perform I/O or wait for blocked resources.

PDF

- Because processes yield the CPU almost immediately after becoming ready, the OS spends more time performing scheduler context switching and managing state queues than running productive application instructions.

PDF

2. System-Level Optimization

- **I/O Optimization & Async Buffering:** Implement Non-blocking asynchronous I/O (e.g., epoll, io_uring, or DMA) and batch small I/O requests. This prevents processes from immediately transitioning to the waiting state upon every minor read/write.

PDF

- **CPU Scheduler Tuning:** Increase the minimum granularity/time-slice for ready processes or group threads using CPU affinity to reduce context switching overhead and keep CPU utilization high.

Q3. Memory Bottleneck Debugging

1. Root Cause Identification

Calculating total system memory requirements:

- **Code:** 5 MB

PDF

- **Stack:** 25 threads $\times$ 1 MB/thread $=$ 25 MB

PDF

- **Heap:** 150 MB

PDF

- **Total Application Footprint:** 5 MB $+$ 25 MB $+$ 150 MB $=$ 180 MB

PDF

Because the memory footprint (180 MB) is significantly smaller than physical RAM (2 GB), the page faults are not caused by physical memory exhaustion. The issue is **High Memory Access Locality Degradation / Excessive Dynamic Allocation (Heap Fragmentation / Cache Invalidation)** leading to continuous soft page faults or dynamic page allocation overhead.

PDF+ 1

2. Proposed Optimizations

- **Optimization 1 (Memory Pooling & Allocation Pre-allocation):** Implement a thread-local memory pool (e.g., jemalloc or custom slab allocators) for heap allocations. Pre-allocating contiguous buffers avoids triggering memory management soft page faults during dynamic heap growth.

PDF

- **Optimization 2 (Optimize Data Locality & Thread Count):** Restructure large data structures into contiguous memory layouts (Array of Structures to Structure of Arrays) to maximize spatial and temporal locality, thereby minimizing page-table invalidation across CPU caches.

Q4. Starvation Debugging

1. Root Cause Identification

Process $\diamond\diamond_5$ suffers from **Indefinite Starvation** due to the static design of Multilevel Queue Scheduling.

PDF

- High-priority queues ($\diamond\diamond_1$ and $\diamond\diamond_2$) are strictly prioritized over

$\diamond\diamond_3$. PDF

- If $\diamond\diamond_1$ (Interactive) or $\diamond\diamond_2$ (System) constantly have active tasks entering their queues, $\diamond\diamond_3$ (Batch, FCFS) will never receive CPU time.

PDF

2. System Optimization (Fairness Enhancement)

Transition the scheduler from a **Multilevel Queue** to a **Multilevel Feedback Queue (MLFQ) with Aging**:

- **Aging Mechanism:** Implement an aging policy where tasks residing in lower priority queues ($\diamond\diamond_3$) for longer than a specific threshold (e.g., 100 ms) automatically get promoted to higher-priority queues ($\diamond\diamond_2$ or $\diamond\diamond_1$).

- **Time-Sliced Priority Allocations:** Implement a weighted CPU time allocation scheme across queues (e.g., 60%CPU time to $\diamond\diamond_1$, $30\%$to $\diamond\diamond_2$, $10\%$to $\diamond\diamond_3$) to guarantee forward progress for $\diamond\diamond_3$regardless of $\diamond\diamond_1$and $\diamond\diamond_2$workloads.

Q5. Context-Switch Overhead Debugging
1. Root Cause Identification

The high context-switch rate ($20{,}000$ switches/sec) is driven by a critical mismatch between scheduler time quantum and execution bursts:

PDF

- **Quantum Misconfiguration:** The Time Quantum (1 ms) is strictly smaller than the average thread burst time (2 ms). Every thread gets preempted forcedly mid-burst, generating at least $10{,}000$ switches/secpurely due to time slice expiration.

PDF

- **Frequent I/O Preemption:** Combined with a "very high" I/O wait frequency, threads self-yield early, introducing an additional $10{,}000+$ context switches/sec.

PDF

2. Proposed Optimizations

- **Optimization 1 (Increase Time Quantum):** Increase the scheduler time quantum from 1 msto 5 ms $-$ 10 ms. This allows threads to complete their average 2 msexecution bursts naturally without undergoing forced CPU preemption.

PDF

- **Optimization 2 (Implement Event-Driven Architecture / Thread Pooling):** Transition from a multi-threaded blocking I/O model to an event-driven architecture (e.g., non-blocking I/O event loops) or reduce active worker threads using an explicit thread pool matching physical CPU core counts.

Here is a complete, line-by-line detailed breakdown and fully documented solution tailored to maximum marks under each rubric criterion (Problem Identification, Debugging, Optimization, Analysis, and Code Quality).

Q1. Deadlock Debugging (Code Trace)

1. Root Cause Identification

The deadlock is caused by a violation of lock ordering across processes, creating a **Circular Wait** condition:

- **Mutual Exclusion:** P_1 and P_2 are unsharable resources.
- **Hold and Wait:** P_1 acquires P_1 and holds it while waiting for P_2; P_2 acquires P_2 and holds it while waiting for P_1.
- **No Preemption:** Resources cannot be forcibly revoked.
- **Circular Wait:** P_1 holds P_1 and waits for P_2 (held by P_2), while P_2 holds P_2 and waits for P_1 (held by P_1).

Thread Trace Execution Timeline:

t = 0 ms : P1 locks R1 | P2 locks R2

t = 100 ms : P1 requests R2 (Blocked, waiting for P2)

P2 requests R1 (Blocked, waiting for P1)

Result : Deadlock achieved

2. Optimization Strategy & Clean Code

To eliminate deadlock without changing application functionality, enforce a strict **Global Lock Ordering Hierarchy** ($P_1 \rightarrow P_2$ across all processes).

C

```
#include <pthread.h>
#include <unistd.h>
#include <stdio.h>

// Shared Mutex Resources
pthread_mutex_t R1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t R2 = PTHREAD_MUTEX_INITIALIZER;

/**
 * @brief Process P1 implementation using standard lock ordering (R1 -> R2)
 */
void* process_P1(void* arg) {
 pthread_mutex_lock(&R1);
 usleep(100000); // 100 ms work window
```

```
 pthread_mutex_lock(&R2);


 // Critical Section Work
 printf("Process P1 executing safely in critical section.\n");
 pthread_mutex_unlock(&R2);
 pthread_mutex_unlock(&R1);
 return NULL;
}


/**
 * @brief Process P2 implementation optimized to match lock ordering of P1 (R1 -> R2)
 */
void* process_P2(void* arg) {
 // OPTIMIZATION: Acquire R1 before R2 to break Circular Wait condition
pthread_mutex_lock(&R1);
 usleep(100000); // 100 ms work window
 pthread_mutex_lock(&R2);


 // Critical Section Work
 printf("Process P2 executing safely in critical section.\n");


 pthread_mutex_unlock(&R2);
 pthread_mutex_unlock(&R1);
 return NULL;
}
```

Q2. CPU Utilization Problem

1. Root Cause Identification

The low CPU utilization (30%) paired with dynamic cycles (ready → waiting → ready

→ waiting) is caused by **Synchronous I/O Blocking and Thrashing Overhead**:

- Tasks run for microscopic time slices before making blocking system calls.
- Context-switching overhead dominates execution cycles, causing the CPU to sit idle in kernel space managing wait queues instead of executing user instructions.

2. Proposed System-Level Optimizations

1. **Non-blocking Asynchronous I/O (epoll / io_uring):** Shift from blocking system calls to event-driven architectures where threads remain in ready/running states while handling multiple network or disk descriptor operations.
2. **Buffer Aggregation & I/O Batching:** Aggregate tiny consecutive read/write requests into contiguous block buffers to reduce system-call frequency.

C

```
#include <sys/epoll.h>
#include <fcntl.h>
#include <unistd.h>

#define MAX_EVENTS 64

/**
 * @brief Non-blocking event loop execution eliminating continuous task
 yields */
void process_events_async(int server_fd) {
 int epoll_fd = epoll_create1(0);
 struct epoll_event ev, events[MAX_EVENTS];

 // Set non-blocking flags
 fcntl(server_fd, F_SETFL, fcntl(server_fd, F_GETFL, 0) | O_NONBLOCK);

 ev.events = EPOLLIN | EPOLLET; // Edge-triggered asynchronous notification
```

```
ev.data.fd = server_fd;
 epoll_ctl(epoll_fd, EPOLL_CTL_ADD, server_fd, &ev);

 // Main execution loop keeps CPU utilized efficiently without frequent wait yields
int nfds = epoll_wait(epoll_fd, events, MAX_EVENTS, 0); // Non-blocking check
 for (int n = 0; n < nfds; ++n) {
 // Execute batch I/O operations
 }
 close(epoll_fd);
}
```

Q3. Memory Bottleneck Debugging

1. Analysis & Problem Identification

Calculated Process Footprint:

- **Code Segment:** 5 MB
- **Stack:** 25 threads $\times$ 1 MB/thread $= 25$ MB
- **Heap:** 150 MB
- **Total Allocation:** $5 + 25 + 150 = 180$ MB

Since total footprint (180 MB) is much lower than total physical RAM (2 GB $= 2048$ MB), page faults are **Soft Page Faults / Heap Fragmentation** caused by frequent unmapped dynamic allocations (minor page faults handled via OS page mapping).

2. Two System-Level Optimizations

1. **Thread-Local Memory Pooling:** Pre-allocate physical memory arenas during application setup to eliminate dynamic runtime kernel page requests.
2. **Data Structure Cache-Line Realignment:** Align structure elements along 64-byte boundaries to reduce cache-miss frequency and eliminate cross-thread false sharing.

C

```
#include <stdlib.h>
#include <pthread.h>
```

```
#define ARENA_SIZE (10 * 1024 * 1024) // 10MB per thread pre-allocation

typedef struct {
 char* memory_arena;
 size_t current_offset;
} LocalPool;

static __thread LocalPool thread_arena;

/**
 * @brief Initialize pool once to eliminate minor page faults at
 runtime */
void init_thread_arena(void) {
 thread_arena.memory_arena = (char*)malloc(ARENA_SIZE);
thread_arena.current_offset = 0;
}

/**
 * @brief O(1) allocation from local pool without trigger page faults
 */
void* fast_pool_alloc(size_t bytes) {
 if (thread_arena.current_offset + bytes > ARENA_SIZE) return NULL;
void* ptr = &thread_arena.memory_arena[thread_arena.current_offset];
thread_arena.current_offset += bytes;
 return ptr;
}
```

Q4. Starvation Debugging

1. Root Cause Identification

Process $$_5$ in Queue $$_3$ suffers from **Starvation** because the scheduler utilizes fixed-priority non-preemptive queue scheduling across levels:

- Tasks in $$_1$ and $$_2$ hold absolute priority over $$_3$.
- $$_3$ (Batch, FCFS) only runs if both $$_1$ and $$_2$ are empty. Continuous interactive traffic prevents $$_5$ from obtaining CPU time slices.

2. Multilevel Feedback Queue (MLFQ) with Aging

Implement an **Aging Mechanism** where long-waiting processes are automatically promoted to higher queues.

C

```
#include <stdio.h>


typedef struct {
 int pid;
 int priority_queue; // Queue 1 (High), Queue 2 (Mid), Queue 3 (Low)
int total_wait_time_ms;
} ProcessControlBlock;


#define MAX_WAIT_THRESHOLD_MS 50


/**
 * @brief Evaluates wait times and promotes starved
 processes */
void apply_scheduler_aging(ProcessControlBlock* pcb) {
 if (pcb->priority_queue > 1 && pcb->total_wait_time_ms >
MAX_WAIT_THRESHOLD_MS) {
 pcb->priority_queue--; // Promote process to higher priority tier
pcb->total_wait_time_ms = 0; // Reset wait time
```

```
 printf("[SCHEDULER] Process %d promoted to Queue %d due to aging.\n",
pcb->pid, pcb->priority_queue);
 }
}
```

Q5. Context-Switch Overhead Debugging

1. Root Cause Identification

The high rate of $20{,}000$ switches/sec is caused by:

1. **Quantum Misconfiguration:** Average execution burst time (2 ms) > Time Quantum (1 ms). Every thread gets preempted forcedly mid-burst.
2. **High I/O Yielding:** High I/O wait frequency forces threads to constantly switch states from running to blocked.

2. Two Optimizations to Reduce Overhead

1. **Quantum Scaling:** Increase time quantum to 5 ms $-$ 10 ms so 2 ms CPU bursts complete naturally without preemption.
2. **CPU Core Affinity Binding:** Bind threads directly to physical CPU cores to preserve CPU L1/L2 cache locality and reduce scheduling migration overhead.

C

```
#define _GNU_SOURCE
#include <pthread.h>
#include <sched.h>

/**
 * @brief Binds thread to specific CPU core to minimize context switch
 costs */
void pin_thread_to_core(pthread_t thread, int cpu_core_id) {
 cpu_set_t cpuset;
 CPU_ZERO(&cpuset);
 CPU_SET(cpu_core_id, &cpuset);
```

```
 // Apply affinity mask
 pthread_setaffinity_np(thread, sizeof(cpu_set_t), &cpuset);
}
```